



UNIVERSITATEA DIN
BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ

SPECIALIZAREA
INFORMATICĂ



Lucrare de licență

Criptanaliza cifrului Speck32/64 utilizând rețele neuronale convoluționale

Absolvent

Lepădatu Teodor

Coordonator științific

Lect. dr. Bucataru Mihai

București, iulie 2026

1 Introducere

Dezvoltarea accelerată a sistemelor cu resurse limitate a condus la apariția unei clase de algoritmi criptografici proiectați special pentru a funcționa eficient în condiții stricte de memorie, consum energetic și putere de calcul. În acest context, criptografia *lightweight* a devenit un domeniu important de cercetare, oferind soluții adaptate unor aplicații precum rețelele de senzori, etichetele RFID, cardurile inteligente sau diverse dispozitive integrate [3, 5].

Un exemplu reprezentativ din această categorie este cifrul *Speck* (introdus în lucrarea [1]), dezvoltat pentru a oferi un compromis între eficiență și securitate. Varianta *Speck 32/64*, analizată în această lucrare, operează pe blocuri de 32 de biți și utilizează o cheie de 64 de biți, fiind destinată în special platformelor cu resurse hardware reduse.

În paralel cu dezvoltarea algoritmilor criptografici, tehnicile bazate pe învățare automată au început să fie explorate și în domeniul criptanalizei. Aceste metode permit identificarea unor regularități statistice subtile în datele criptografice și pot completa abordările clasice de analiză.

Lucrarea de față investighează utilizarea rețelelor neuronale convoluționale în criptanaliza cifrului *Speck 32/64*, prezentat pe larg în secțiunea 2. În secțiunea 3 sunt analizate mai multe strategii de antrenare și evaluare a modelelor precum și o serie de optimizări menite să îmbunătățească eficiența procesului de căutare a cheii. Rezultatele obținute și prezentate în secțiunea 4 evidențiază atât potențialul tehnicilor bazate pe învățare profundă în criptanaliză, cât și potențialele limite ale acestora în scenarii cu un număr ridicat de runde de criptare.

2 Descrierea algoritmului de criptare *Speck 32/64*

2.1 Parametri

Definiție 2.1 (Parametrii *Speck 32/64*). *Speck* este un cifru bloc de tip ARX

(Addition, Rotation, XOR) care are următorii parametri:

- **word size** = 16 biți
- **block** = (L, R) = cuvântul criptat (care are 32 de biți) este împărțit în două subcuvinte de 16 biți
- **key** = număr pe 64 de biți
- **rotations** = numărul de biți roțiți la dreapta sau la stânga în funcția de criptare (valorile folosite sunt $\alpha = 7$ pentru rotația dreapta pe L și $\beta = 2$ pentru rotația stânga pe R)
- **number of rounds** = numărul de subchei în care este împărțită cheia inițială

2.2 Funcția de criptare

Definiție 2.2 (Operații elementare). Definim:

- $ROR(L, \alpha)$ rotația la dreapta în L cu α biți
- $ROL(R, \beta)$ rotația la stânga în R cu β biți
- K subcheia runde curente, derivată din cheia inițială, având lungimea exactă de w biți
- $\oplus$ operația XOR pe biți
- $\equiv_{2^w}$ congruența modulo 2^w

Acum, pentru **criptarea** mesajului folosim:

$$f_K(L, R) = (L', R')$$

unde:

$$L' = ((ROR(L, \alpha) + R) \bmod 2^w) \oplus K$$

$$R' = ROL(R, \beta) \oplus L'$$

Propoziție 2.1 (Funcția de decriptare). Pentru **decriptare** folosim inversa funcției f_K :

$$f_K^{-1}(L', R') = (L, R)$$

unde:

$$R = ROR(R' \oplus L', \beta)$$

$$L = ROL(((L' \oplus K) - R) \bmod 2^w, \alpha)$$

Demonstrație. Demonstrăm că decriptarea cu această funcție readuce într-adevăr blocul inițial.

Considerăm un cuvânt de lungime w în biți, pe care îl vom roti cu α biți la dreapta, respectiv β biți la stânga. Fie K subcheia de la runda respectivă de criptare. Fie funcția în runda curentă de criptare f_K care mapează (L, R) în (L', R') cu:

$$\begin{cases} L' = ((ROR(L, \alpha) + R) \bmod 2^w) \oplus K \\ R' = ROL(R, \beta) \oplus L' \end{cases}$$

Considerăm maparea g_K :

$$\begin{cases} \tilde{R} = ROR(R' \oplus L', \beta) \\ \tilde{L} = ROL(((L' \oplus K) - \tilde{R}) \bmod 2^w, \alpha) \end{cases}$$

Vrem să demonstrăm că g_K este inversa lui $f_K \iff g_K \circ f_K = id$ și $f_K \circ g_K = id$.

Demonstrăm că $g_K(f_K(L, R)) = (L, R)$:

Fie $(L', R') = f_K(L, R)$. Considerăm $S := R' \oplus L'$. Avem că $R' = ROL(R, \beta) \oplus L'$, deci:

$$S = (ROL(R, \beta) \oplus L') \oplus L' = ROL(R, \beta)$$

Dar $\tilde{R} = ROR(S, \beta) = ROR(ROL(R, \beta), \beta) = R$, deci $\tilde{R} = R$.

Considerăm acum $U := ((L' \oplus K) - \tilde{R}) \bmod 2^w$. Din $L' = ((ROR(L, \alpha) + R) \bmod 2^w) \oplus K$ avem $L' \oplus K \equiv_{2^w} ROR(L, \alpha) + R$, de unde $U \equiv_{2^w} (ROR(L, \alpha) + R) - \tilde{R}$.

Dar cum $R = \tilde{R}$, rezultă $U \equiv_{2^w} ROR(L, \alpha)$.

Așadar, $\tilde{L} = ROL(U, \alpha) = ROL(ROR(L, \alpha), \alpha) = L \Rightarrow L = \tilde{L}$. Deci $g_K \circ f_K = id$.

Demonstrăm acum că $f_K(g_K(L', R')) = (L', R')$:

Fie $(\tilde{L}, \tilde{R}) = g_K(L', R')$, unde $\tilde{R} = ROR(R' \oplus L', \beta)$ și $\tilde{L} = ROL(((L' \oplus K) - \tilde{R}) \bmod 2^w, \alpha)$.

Vrem să arătăm că $f_K(\tilde{L}, \tilde{R}) = (L', R')$.

Fie $\tilde{L}' = ((ROR(\tilde{L}, \alpha) + \tilde{R}) \bmod 2^w) \oplus K$. Din construcția lui $\tilde{L}$, avem $ROR(\tilde{L}, \alpha) \equiv_{2^w} (L' \oplus K) - \tilde{R}$, de unde $ROR(\tilde{L}, \alpha) + \tilde{R} \equiv_{2^w} L' \oplus K$, deci $\tilde{L}' = ((L' \oplus K) \bmod 2^w) \oplus K = L'$.

Fie acum $\tilde{R}' = ROL(\tilde{R}, \beta) \oplus \tilde{L}'$. Avem $\tilde{R} = ROR(R' \oplus L', \beta)$, deci $ROL(\tilde{R}, \beta) = R' \oplus L'$, de unde $\tilde{R}' = (R' \oplus L') \oplus L' = R'$.

Așadar, $f_K(g_K(L', R')) = (L', R')$, deci $f_K \circ g_K = id$. $\square$

3 Atacul pentru decriptare

Atacatorul vede ciphertexte criptate în același mod, fără a cunoaște cheia folosită pentru criptarea lor. El vrea să găsească ultima subcheie cu care s-a criptat un mesaj. Odată ce găsește ultima cheie, repetă procesul până descoperă toată cheia secretă. Toate metricile de evaluare prezentate vor reflecta capacitatea algoritmilor de a returna această ultimă subcheie. În cele ce urmează, abordarea propusă de Gohr [4] va constitui reperul metodologic pentru definirea problemei, generarea datelor de antrenare și evaluarea performanței modelului.

3.1 Rețeaua neuronală convoluțională (CNN)

3.1.1 Definirea problemei

Construim o rețea neuronală convoluțională (CNN) care are rolul de *neural distinguisher*. Rețeaua va returna o probabilitate $p \in [0, 1]$ pentru a răspunde la întrebarea: "Provine perechea de texte cifrate (C_1, C_2) din criptarea a două texte clare care respectă o anumită diferență fixă?". În mod complementar, $1 - p$ va reprezenta probabilitatea ca perechea respectivă să fie formată din secvențe de biți complet aleatoare.

3.1.2 Generarea datelor de antrenare

Pentru a antrena rețeaua folosim perechi de texte clare care diferă printr-o valoare fixă XOR, notată $\Delta P = (\Delta L, \Delta R)$. Procesul de generare a datelor pozitive (reale) presupune criptarea acestor perechi:

$$(L, R) \xrightarrow{\text{encrypt}} C_1$$
$$(L \oplus \Delta L, R \oplus \Delta R) \xrightarrow{\text{encrypt}} C_2$$

Astfel, un eșantion valid pentru rețea va fi format din combinarea perechilor de texte cifrate obținute: (C_1, C_2) . Setul de date complet este obținut echilibrând clasele: o jumătate este formată din exemple pozitive (perechi reale de criptare cu diferența dată), iar cealaltă jumătate din exemple negative (în care al doilea text cifrat C_2 este înlocuit cu o valoare generată uniform aleator).

3.1.3 Arhitectura rețelei

Arhitectura propusă de Gohr [4] este o rețea convoluțională reziduală formată dintr-un residual tower și un prediction head, arhitectură ce a fost simplificată și îmbunătățită pentru a reduce numărul de parametri fără pierderea acurateței în lucrarea [7].

Structura Straturilor

- **Input:** Un tensor tridimensional cu 3 canale, corespunzător caracteristicilor extrase din perechea (C_1, C_2) . Dimensiunea spațială este de 16 biți (conform arhitecturii Speck32/64).
- **Blocul rezidual:** Rețeaua integrează *depth* blocuri reziduale succesive. Fiecare bloc efectuează:
 - **Conv1d** (extindere de la 3 la 32 de canale, kernel = 3, padding = 1).
 - **BatchNorm1d** urmat de funcția de activare neliniară **ReLU**.
 - **Conv1d** (contractare de la 32 înapoi la 3 canale, kernel = 3, padding = 1).
 - **BatchNorm1d** urmat de **ReLU**.
 - O conexiune reziduală care adună intrarea originală a blocului la ieșirea acestuia, pentru a preveni vanishing gradient.
- **Prediction head:** Ieșirea turnului rezidual, de formă $(3, 16)$, este aplati-zată într-un vector de 48 de elemente și trecută prin straturi fully connected:
 - **Linear** ($48 \rightarrow 64$ de neuroni), **BatchNorm1d**, **ReLU**.
 - **Linear** ($64 \rightarrow 64$ de neuroni), **BatchNorm1d**, **ReLU**.
 - **Linear** ($64 \rightarrow 1$ neuron), urmat de funcția **Sigmoid** pentru a mapa ieșirea în probabilitatea binară dorită.

3.1.4 Parametrii și strategia de antrenare

Vom antrena modele pentru 5, 6 (având $depth = 10$) și 7 runde de criptare (având $depth = 1$), iar procesul de antrenare utilizează următoarea configurație hiperparametrică:

- **Funcția de pierdere:** `MSELoss`.
- **Optimizator:** Adam, cu o regularizare $L2$ și weight decay de 10^{-5} .
- **Rata de învățare:** Planificatorul `OneCycleLR`, cu o rată maximă de 10^{-3} .
- **Setul de date:** 10^7 eșantioane în total ($9 \cdot 10^6$ pentru antrenare, 10^6 pentru validare).
- **Batch size:** 5000 de exemple per lot.
- **Durata:** Antrenarea rulează pentru 200 de epoci, reținându-se starea modelului cu cea mai bună acuratețe pe setul de validare.

Timpul total de antrenare a celor 3 rețele este de aproximativ 64 de ore folosind un *CPU i7 11th gen*, un *GPU GTX 1650* (cu *4GB VRAM*) și *16 GB RAM*. Același sistem a fost folosit și pentru metodele de atac ce vor fi prezentate ulterior.

3.1.5 Rezultatele antrenării

Tabela 1: Acuratețea modelului pe setul de validare

Model	Acuratețe (%)
5 runde	92.74
6 runde	78.79
7 runde	55.14
8 runde	≈ 50.00

Din tabelul de mai sus se poate observa că dacă am încerca să folosim aceeași strategie de antrenare pentru modele de 8 sau mai multe runde, acuratețea va fi în jurul valorii de 50 %, deci echivalent cu alegerea aleatorie a clasei, rezultat care nu poate fi folosit de niciunul dintre algoritmi de criptanaliză prezentați mai jos. Pentru testarea pe 8 runde am folosit tot $depth = 1$, similar cu scenariul pe 7 runde.

3.2 Utilizarea probabilităților CNN

Odată antrenat, neural distinguisher-ul (DND-ul) nu este folosit izolat, ci ca o componentă centrală în faza de recuperare a subcheii (inferența rețelei pe date parțial decriptate). În continuare, vom prezenta două metode de agregare a probabilităților pentru a determina subcheia corectă.

3.2.1 Sum of Logits

Deoarece acuratețea rețelei pentru o singură pereche de texte cifrate este limitată, se utilizează structuri de texte cifrate generate pe baza unor biți neutri. Răspunsurile rețelei pentru toate perechile din structură sunt agregate pentru a formula un scor de încredere pentru fiecare cheie candidată.

Definiție 3.1 (Notății matematice pentru SoL).

- $f_0(X) = P(real|X)$: probabilitatea returnată de CNN ca datele de intrare X să provină dintr-o criptare reală.
- $X_i(K) = f^{-1}(C_i, K)$: rezultatul decriptării parțiale (cu o rundă) a perechii de texte cifrate C_i folosind subcheia candidată K .
- $p_i(K) = f_0(X_i(K))$: probabilitatea estimată de rețea pentru perechea i decriptată cu cheia K .
- $l_i(K) = \log_2 \left(\frac{p_i(K)}{1-p_i(K)} \right)$: transformarea probabilității în *log-odds*.

Descriere:

Presupunem că dispunem de n perechi de texte cifrate $(C_{i1}, C_{i2}), i = \overline{1, n}$, obținute dintr-o structură, și cunoaștem lungimea ultimei subchei. Pentru fiecare cheie candidată K din spațiul de chei aferent, decriptăm parțial cele n perechi. CNN-ul evaluează fiecare rezultat, oferind o probabilitate $p_i(K)$. Scorul total pentru subcheia candidată K se calculează prin însumarea valorilor log-odds:

$$S(K) = \sum_{i=1}^n \log_2 \left(\frac{p_i(K)}{1-p_i(K)} \right)$$

Valoarea maximă a scorului $S(K)$ va indica subcheia cea mai probabilă.

Fundamentarea teoretică a metodei

Ipoteza 1 (Condiții de optimalitate teoretică). Această abordare este optimă sub două presupuneri stricte:

1. CNN-ul este *Bayes-optimal*, adică probabilitatea prezisă reflectă perfect distribuțiile reale:

$$P(real|X) = \frac{P_{real}(X)}{P_{real}(X) + P_{random}(X)}$$

unde $P_{real}(X)$ este densitatea de probabilitate sub ipoteza că intrarea provine din distribuția cifrului, iar $P_{random}(X)$ este densitatea sub ipoteza unei distribuții uniforme.

2. Cele n exemple decriptate parțial $X_i(K)$ sunt independente condiționat de cheia K .

Pe baza primei presupuneri, deducem că:

$$\frac{f_0(X)}{1 - f_0(X)} = \frac{P_{real}(X)}{P_{random}(X)} \iff \log_2 \left(\frac{f_0(X)}{1 - f_0(X)} \right) = \log_2 \left(\frac{P_{real}(X)}{P_{random}(X)} \right)$$

Prin urmare, formula scorului devine echivalentă cu însumarea logaritmilor verosimilității (log-likelihood):

$$\forall i, K : l_i(K) = \log_2 \left(\frac{p_i(K)}{1 - p_i(K)} \right) = \log_2 \left(\frac{P_{real}(X_i(K))}{P_{random}(X_i(K))} \right)$$

Dacă rețeaua este Bayes-optimală, maximizarea lui $S(K)$ este o decizie teoretic optimă pentru clasificarea secvențelor independente. În practică, deoarece CNN-ul este doar o aproximare a distribuției ideale, metoda oferă un doar estimator empiric, dar destul de robust având în vedere simplitatea acestui algoritm.

Evaluarea algoritmului

Pentru a ataca un sistem pe n runde, vom folosi modelul antrenat pe $n - 1$ runde. Vom considera că atacul a avut succes doar atunci când cheia adevărată se află printre un top de 32 de chei considerate de algoritm ca fiind cele mai probabile să fie cheia reală. Rezultatele se pot observa în acest tabelul de mai jos.

Tabela 2: Performanța atacului Sum of Logits (Top 32)

Atac	Rată de succes (%)	Loc mediu cheia reală
6 runde	100.0	1.50
7 runde	100.0	1.60
8 runde	100.0	5.10

3.2.2 Bayesian Key Search

Când decriptarea de probă se face pentru o singură rundă, ipoteza randomizării pentru chei greșite eșuează adesea, mai ales în cazul cifrurilor ușoare precum Speck32/64. Pentru a rezolva această problemă și a eficientiza căutarea, se utilizează algoritmul *Bayesian Key Search* (BKS) prezentat în lucrarea [4], îmbunătățit prin preluarea și adaptarea acestuia pentru a garanta păstrarea cheilor optime în lucrarea [7].

Precalcularea profilului (WKRP)

Se generează un profil al răspunsului rețelei pentru chei greșite (WKRP - *Wrong Key Response Profile*). Pentru diverse diferențe dintre cheia reală și cheia de

test ($\Delta k = k_i \oplus k$), se evaluează textele cifrate decriptate parțial. Transformând rezultatele DND în log-odds, obținem media $\mu_{\Delta k}$ și deviația standard $\sigma_{\Delta k}$ pentru fiecare diferență Δk posibilă.

Algoritmul de căutare iterativă

Spre deosebire de forța brută (evaluarea întregului spațiu de chei), algoritmul BKS îmbunătățit execută ℓ iterații pentru a rafina succesiv un set restrâns de candidați, proces care se desfășoară după cum urmează:

Pasul 1. Se pornește cu un set S format din n_{cand} chei candidate. Pentru a preveni pierderea cheii reale din cauza fluctuațiilor statistice, dacă există o cheie optimă globală determinată în pașii sau loturile anterioare, aceasta este reținută forțat în setul curent de candidați.

Pasul 2. Pentru fiecare cheie candidată $k_i \in S$ și fiecare pereche de texte cifrate j din structură, decriptăm o rundă, trecem rezultatul prin DND pentru a obține probabilitatea v_{j,k_i} , iar apoi calculăm log-odds:

$$z_{j,k_i} = \log_2 \left(\frac{v_{j,k_i}}{1 - v_{j,k_i}} \right)$$

Pasul 3. Se calculează scorul mediu log-odds pentru fiecare cheie candidată k_i :

$$m_{k_i} = \frac{1}{n_{cts}} \sum_{j=0}^{n_{cts}-1} z_{j,k_i}$$

Pasul 4. Parcurgând întreg spațiul de chei posibile $k \in \mathcal{K}$, se calculează un scor de penalizare $\lambda(k)$, reprezentând distanța euclidiană ponderată:

$$\lambda(k) = \sum_{i=0}^{n_{cand}-1} \frac{(m_{k_i} - \mu_{k_i \oplus k})^2}{\sigma_{k_i \oplus k}^2}$$

Pasul 5. Se actualizează setul S reținând cele n_{cand} chei k care minimizează scorul $\lambda(k)$ și se revine la **Pasul 1** pentru următoarea iterație.

Fundamentarea teoretică a metodei

Ipoteza 2 (Condiții statistice BKS). Eficiența metodei BKS se bazează pe următoarele presupuneri:

1. Mediile log-odds empirice (m_{k_i}) urmează o distribuție normală dictată de diferența de cheie.
2. Parametrii μ și σ precalculați în tabelul WKRP reflectă fidel distribuția reală.

Considerând că media log-odds-urilor obținute cu cheia k_i este distribuită normal în raport cu profilul cheii reale k , i.e.

$$m_{k_i} \sim \mathcal{N}(\mu_{k_i \oplus k}, \sigma_{k_i \oplus k}^2)$$

Funcția de verosimilitate (likelihood) pentru vectorul de medii observate m , condiționată de cheia corectă k , devine:

$$P(m|k) = \prod_{i=0}^{n_{cand}-1} \frac{1}{\sqrt{2\pi\sigma_{k_i \oplus k}^2}} e^{-\frac{(m_{k_i} - \mu_{k_i \oplus k})^2}{2\sigma_{k_i \oplus k}^2}}$$

Aplicând teorema lui Bayes (cu o distribuție a priori uniformă peste spațiul cheilor $P(k)$) obținem $P(k|m) \approx P(m|k)$. Trecând în domeniul logaritmico pentru a evita instabilitatea numerică și ignorând termenii constanți, maximizarea probabilității $\log_2 P(k|m)$ devine echivalentă cu minimizarea metricii noastre de eroare $\lambda(k)$:

$$\lambda(k) = \sum_{i=0}^{n_{cand}-1} \frac{(m_{k_i} - \mu_{k_i \oplus k})^2}{\sigma_{k_i \oplus k}^2}$$

Evaluarea algoritmului

Am folosit aceeași metodă de evaluare ca pentru Sum of logits și am obținut următoarele rezultate:

Tabela 3: Performanța atacului Bayesian Key Search (Top 32)

Atac	Rată de succes (%)	Loc mediu cheie reală
6 runde	100.0	1.00
7 runde	100.0	1.00
8 runde	20.0	2.50

Chiar dacă rata de succes scade, avantajul principal al acestui algoritim este locul mediu al cheii reale, care a crescut considerabil față de *sum of logits*.

3.3 Ajustarea fină a modelului (Fine-Tuning cu exemple dificile)

Deși rețeaua neuronală descrisă anterior obține o acuratețe ridicată pe setul de date standard, performanța acesteia poate scădea în faza de atac (la recuperarea cheii). În generarea datelor standard, exemplele negative sunt create prin înlocuirea unui text cifrat valid cu date complet aleatoare. Totuși, în practică, în timpul atacului, rețeaua evaluează texte cifrate care au fost decriptate cu o subcheie candidată greșită. Aceste decriptări eronate nu produc un zgomot perfect aleator, ci păstrează anumite corelații structurale specifice cifrului, fenomen care slăbește distincția modelului. Pentru a rezolva această problemă, vom face

un fine-tuning al modelului folosind exemple negative dificile, ce imită aceste scenarii.

3.3.1 Generarea exemplelor negative dificile

Când generăm aceste exemple, vom simula exact scenariul întâlnit la căutarea cheii. Mai întâi, se generează și se criptează perechi de texte pentru R runde folosind cheile corecte, iar apoi, pentru un eșantion negativ, perechea obținută este criptată încă o rundă folosind subcheia corectă. Imediat după, perechea este decriptată o rundă, dar de această dată folosind o subcheie aleatoare.

Astfel, obținem un set de date care imită mai bine scenariile pe care modelul le va întâlni la inferență.

3.3.2 Rezultatele fine-tuning-ului

Am antrenat modelele pentru încă 20 de epoci folosind astfel de exemple negative. După finalizarea antrenării, acuratețea modelelor pentru perechi de acest tip a crescut cu aproximativ 1%.

3.4 Modele propuse și algoritmi hibridi

Pentru a maximiza atât viteza de execuție, cât și rata de succes a atacului de decriptare, am implementat și evaluat o serie de strategii derivate. Acestea pleacă de la utilizarea directă a modelului fine-tuned și ajung până la algoritmi hibridi. Algoritmii hibridi combină avantajele de pre-filtrare ale metodei *Sum of Logits* (SoL) cu precizia rafinată a *Bayesian Key Search* (BKS), utilizând simultan cele două tipuri de rețele neuronale: modelul de bază (antrenat standard) și modelul cu ajustare fină (*fine-tuned*, notat FT).

Aceste strategii pot fi clasificate astfel:

- **Fine-Tuned BKS:** Această abordare reprezintă o îmbunătățire directă a algoritmului de bază. Execută algoritmul Bayesian Key Search pe întreg spațiul de chei ($2^{16} = 65536$ posibilități), dar folosește exclusiv modelul FT și profilul WKRP corespunzător acestuia, înlocuind modelul de bază.
- **SoL → Original BKS:** Evaluarea întregului spațiu de chei folosind direct BKS este o operațiune costisitoare computațional. Prin această metodă hibridă, utilizăm inițial tehnica SoL pe modelul de bază pentru a aproxima foarte rapid și a extrage doar un top restrâns de candidați (64 de chei). Setul obținut devine spațiul de căutare exclusiv pentru algoritmul BKS original, reducând masiv timpul de execuție, limitând în același timp numărul de candidați eronați care ar fi putut cauza rezultate fals-pozitive.
- **SoL → Fine-Tuned BKS:** Această tehnică respectă același principiu de restrângere a spațiului de căutare. După identificarea celor mai probabile 64 de chei folosind SoL și modelul de bază, rafinarea finală se face aplicând BKS exclusiv cu modelul FT. Metoda combină capacitatea de generalizare

a modelului de bază cu acuratețea superioară a modelului FT în fața exemplurilor negative dificile.

- **Ensemble BKS:** În cadrul acestei implementări de tip BKS, în locul inferenței pe un singur model, neural distinguisher-ul acționează ca un ansamblu format din ambele modele (cel standard și cel FT). Răspunsurile acestora sunt ponderate pe baza inversului pătratelor funcției de pierdere obținute în procesul de validare:

$$w_i = \frac{1}{\text{loss}_i^2}, \quad i \in \{1, 2\}$$

Ponderile normalizate devin astfel:

$$\alpha_1 = \frac{w_1}{w_1 + w_2}, \quad \alpha_2 = \frac{w_2}{w_1 + w_2}$$

Probabilitatea agregată finală (p_{final}), care va fi ulterior transformată în log-odds pentru algoritmul bayesian, rezultă din suma ponderată a probabilităților individuale prezise de cele două modele:

$$p_{final} = \alpha_1 \cdot p_{base} + \alpha_2 \cdot p_{FT}$$

- **SoL → Ensemble BKS:** Cea mai complexă arhitectură hibridă testează toate cele 65536 de chei folosind SoL și modelul de bază, iar cele 64 de chei care supraviețuiesc acestei etape de pre-filtrare sunt transmise mai departe către modelul de căutare *Ensemble BKS* pentru determinarea optimului global.

3.5 Optimizarea algoritmului BKS

Deși algoritmul BKS aduce îmbunătățiri majore în faza de recuperare a cheii, calculul inițial al profilului WKRP (*Wrong Key Response Profile*) este o operațiune extrem de costisitoare din punct de vedere computațional. Pentru a construi acest profil de distribuții, rețeaua neuronală trebuie să evalueze milioane de perechi de texte cifrate pentru absolut toate cele $2^{16} = 65536$ de diferențe posibile de cheie (Δk). Pentru că acest profil rămâne identic pentru orice atac făcut de același model pe același număr de runde, am implementat un mecanism de caching offline. În loc să recalculăm profilul pentru fiecare nouă instanță sau scenariu de atac, tabelele WKRP (vectorii de medii μ și deviații standard σ) sunt generate o singură dată pentru fiecare model antrenat (atât cel de bază, cât și cel fine-tuned) și sunt serializate pe disc sub formă de fișiere. În faza de atac propriu-zisă doar se încarcă profilul în VRAM, operație mult mai rapidă decât calcularea profilului, dar și decât algoritmul *sum of logits* pentru care o astfel de precalculare este imposibilă.

Astfel, algoritmul BKS rulează inferența rețelei neuronale exclusiv pentru un set activ foarte mic de chei candidate ($n_{cand} = 64$). Pentru a explora și a evalua restul spațiului de 65536 de chei, BKS nu mai apelează rețeaua neuronală. În

schimb, folosește valorile μ și σ din cache-ul WKRP pentru a calcula distanța euclidiană ponderată $\lambda(k)$:

$$\lambda(k) = \sum_{i=0}^{n_{cand}-1} \frac{(m_{k_i} - \mu_{k_i \oplus k})^2}{\sigma_{k_i \oplus k}^2}$$

Această evaluare bayesiană se reduce la operații matematice vectorizate elementare pe tensori, care sunt executate aproape instant pe arhitectura masiv paralelă a unui GPU, ocolind complet necesitatea unor noi apeluri ale CNN-ului.

4 Evaluarea și rezultatele metodelor de criptanaliză

Pentru a compara eficiența metodelor propuse, evaluarea a fost realizată printr-un cadru de testare automatizat care simulează scenarii de atac pe 6, 7 și 8 runde.

4.1 Metodologia de evaluare

Procesul de evaluare funcționează printr-o buclă continuă, în care fiecare iterație reprezintă o nouă provocare. Pentru fiecare provocare este generată o subcheie țintă generată aleatoriu (reprezentând obiectivul atacului ce trebuie recuperat) și un număr specific de structuri de texte clare și cifrate (32 de structuri pentru atacul pe 6 runde, 64 de structuri pentru 7 runde și 128 de structuri pentru 8 runde).

În cadrul fiecărei iterații, textele cifrate sunt transmise secvențial către șase metode distincte de recuperare a cheii:

- **M1 (Original BKS):** Metoda de bază, folosind algoritmul BKS cu modelul antrenat standard (adică metoda descrisă în lucrarea [4]).
- **M2 (Fine-Tuned BKS):** Algoritmul BKS evaluat exclusiv cu modelul fine-tuned.
- **M3 (SoL → Original BKS):** Reducerea spațiului de căutare la 64 de candidați folosind metoda *Sum of Logits* (SoL) și evaluarea acestora cu BKS-ul original.
- **M4 (SoL → Fine-Tuned BKS):** Reducerea spațiului prin SoL, urmată de evaluarea BKS folosind modelul ajustat fin.
- **M5 (Ensemble BKS):** Algoritmul BKS utilizând un ansamblu format din ambele modele (standard și ajustat).
- **M6 (SoL → Ensemble BKS):** Spațiu redus prin SoL la primele 64 de chei, evaluat ulterior cu metoda Ensemble BKS.

4.2 Metrice înregistrate și criteriul de oprire

Pentru fiecare metodă și fiecare provocare, algoritmul măsoară timpul de execuție și verifică dacă subcheia prezisă se potrivește perfect cu subcheia țintă reală. Pe baza acestor date, se calculează și se actualizează dinamic acuratețea globală (numărul de predicții corecte raportat la numărul total de rulări) și timpul mediu de execuție. Spre deosebire de evaluările anterioare, vom considera că atacul a avut succes doar dacă cheia corectă este prima în topul cheilor prezise de algoritm. Astfel, acuratețea va fi considerabil mai mică decât în analiza anterioară.

Pentru a asigura o evaluare relevantă din punct de vedere statistic, mediul de testare impune un număr de minimum 10 rulări înainte de a verifica vreo condiție de dominanță. Bucla infinită de evaluare se oprește doar în momentul în care cel puțin una dintre metodele propuse (M2 – M6) demonstrează o dominanță clară asupra metodei de bază (M1). Această dominanță este definită prin îndeplinirea uneia dintre următoarele două condiții:

- **Acuratețe superioară:** Acuratețea metodei propuse este strict mai mare decât acuratețea metodei de bază (M1).
- **Dominanță la viteză:** Metoda propusă obține o acuratețe mai mare sau egală cu cea a metodei M1 (ambele având o acuratețe strict mai mare ca 0), dar înregistrează un timp mediu de execuție strict mai mic.

În momentul în care oricare dintre aceste criterii de succes este atins, rularea scenariului se oprește, mediul afișează rezumatul final cu timpii și acuratețile tuturor celor 6 metode, iar întregul istoric al seturilor de date aferente (până la o limită fixă de 1000 de provocări) este salvat automat pe disc.

4.3 Rezultatele atacurilor

Pentru a valida performanța metodelor propuse, bucla de testare a fost rulată pentru cele trei scenarii distincte de atac (6, 7 și 8 runde). În toate cele trei cazuri, algoritmi hibridi și cei bazați pe modelul fine-tuned au demonstrat o superioritate clară față de metoda originală de referință (M1 - Orig BKS), rezultatele menținând un tipar consistent de dominanță. Această metodă de evaluare fiind constructivă, am găsit și un set de date în care cel puțin unul dintre algoritmi propuși este mai performant decât algoritmul de referință. Totuși, orice rulare a notebook-ului aferent comparației dintre algoritmi va genera un set de date pentru care prima metodă este inferioară cel puțin uneia dintre celelalte metode propuse.

4.3.1 Analiza acurateței

În primele două scenarii de atac (6 și 7 runde), metodele M2 (FT BKS), M4 (SoL → FT BKS) și M5 (Ensemble BKS) au atins rapid o acuratețe de 100%, depășind performanța de 90% a algoritmului de bază (M1). Mai mult, în cel de-al treilea scenariu (8 runde), considerat un caz extrem din cauza degradării

probabilităților și a zgomotului masiv din date, metodele pur bayesiene (M1, M2, M5) au înregistrat o rată de succes de 0%. În contrast direct, metodele care folosesc *Sum of Logits* pentru pre-filtrare (M3, M4, M6) au reușit să recupereze cheia corectă în 20% din cazuri, demonstrând o robustețe net superioară a arhitecturilor hibride.

4.3.2 Analiza timpului de execuție

Diferențele masive ale timpului de execuție confirmă avantajul teoretic al mecanismului de caching offline explicat anterior. Metodele BKS directe (M1 și M2) sunt extrem de rapide, finalizând o provocare în sub o secundă. Metoda M5 (*Ensemble BKS*) necesită aproximativ dublul acestui timp, o creștere logică și eficientă, având în vedere că prelucrează simultan răspunsurile a două rețele neuronale diferite.

Pe de altă parte, metodele care integrează componenta SoL (M3, M4, M6) sunt constrânse de necesitatea de a efectua inferența CNN pe întregul spațiu de 65536 de chei. Acest aspect duce la timpi de execuție semnificativ mai mari, ilustrând perfect compromisul viteză-complexitate, dar cu o performanță mai ridicată în unele cazuri. Rezultatele pot fi analizate comparativ în tabelele de mai jos.

Metoda M2 rulează constant cu 0.01 – 0.03 secunde mai rapid decât M1. Aceasta nu este o diferență algoritmică, ci un artefact PyTorch: M1 suportă latența inițială de transfer CUDA și overhead-ul alocării memoriei VRAM, în timp ce M2 refolosește instantaneu resursele lăsate în cache. Performanța reală este echivalentă, lucru demonstrat și de rularea pe platforme cloud (Google Colab, Kaggle), unde acest decalaj de warm-up dispăre complet datorită pre-alocării agresive a memoriei.

Tabela 4: Rezultate atac 6 runde (DND 5r)

Metrică	M1	M2	M3	M4	M5	M6
Acuratețe (%)	90.0	100.0	100.0	100.0	100.0	100.0
Timp Mediu (s)	0.54	0.52	257.65	155.67	1.06	232.06

Tabela 5: Rezultate atac 7 runde (DND 6r)

Metrică	M1	M2	M3	M4	M5	M6
Acuratețe (%)	90.0	100.0	100.0	100.0	100.0	100.0
Timp Mediu (s)	0.78	0.77	342.34	261.34	1.52	321.23

Tabela 6: Rezultate atac 8 runde (DND 7r)

Metrică	M1	M2	M3	M4	M5	M6
Acuratețe (%)	0.0	0.0	20.0	20.0	0.0	20.0
Timp Mediu (s)	0.75	0.72	305.51	242.12	1.48	255.42

5 Concluzii și direcții viitoare de cercetare

Această lucrare a demonstrat eficiența rețelelor neuronale convoluționale în criptanaliza cifrului Speck 32/64, aducând îmbunătățiri semnificative procesului de recuperare a cheii secrete prin optimizări arhitecturale și algoritmice. Codul sursa dezvoltat pentru antrenarea modelelor și mediul de evaluare a algoritmilor este disponibil public pe GitHub la adresa: https://github.com/TeodorLepadatu/Bachelor_thesis.

Prima concluzie este că fine-tuning-ul pe exemple negative dificile este esențială. Aceasta forțează rețeaua să elimine corelațiile false, reducând drastic rezultatele fals-pozitive.

În al doilea rând, optimizarea prin caching offline a profilului WKRP elimină necesitatea inferenței repetate, transformând algoritmul *Bayesian Key Search* (BKS) într-un proces mult mai rapid decât abordarea *Sum of Logits* (SoL).

Principala contribuție o reprezintă algoritmi hibrizi propuși (M2 – M6), care oferă un compromis ideal între viteză și acuratețe. Pentru atacuri pe 6 și 7 runde, metodele *FT BKS* și *Ensemble BKS* obțin o acuratețe de 100% într-un timp de maxim 2 secunde. Pentru scenariul de 8 runde, unde metodele clasice eșuează complet (0% succes) pe setul de date găsit, utilizarea pre-filtrării SoL combinată cu analiza *Ensemble* a reprezentat singura soluție viabilă, recuperând cheia în 20% din cazuri. Deși rezultatele obținute au demonstrat o eficiență remarcabilă pentru atacurile de până la 7 runde, extinderea acestor tehnici către 8 sau mai multe runde a scos la iveală limitările abordărilor standard de antrenare. Așa cum s-a observat în faza de evaluare, un model antrenat de la zero pentru 8 runde nu reușește să depășească pragul de 50% acuratețe (echivalentul unei ghiciri aleatoare).

O direcție promițătoare pentru depășirea acestui blocaj este utilizarea tehnicilor de *Transfer Learning* și *Curriculum Learning*. Aron Gohr [4] propune utilizarea unui model pre-antrenat pe un număr inferior de runde (de exemplu, 7 runde) ca punct de plecare sau ca rețea profesor pentru a ghida antrenarea unui model pe 8 runde. Deși experimentele inițiale din literatură arată că această metodă nu funcționează perfect din prima încercare, rafinarea procesului de transfer de cunoștințe reprezintă un punct de plecare vital pentru viitor.

Pe lângă extinderea numărului de runde, cercetările viitoare se pot concentra pe următoarele aspecte:

- **Explorarea unor noi arhitecturi neuronale:** Trecerea de la arhitectura convoluțională reziduală folosită în prezent către arhitecturi moderne bazate pe mecanisme de atenție (cum ar fi *Transformers*). Spre deosebire

de filtrele convoluționale, care sunt limitate la un câmp receptiv local, mecanismele de self-attention ar putea capta mult mai eficient dependențele neliniare și globale dintre biții textelor cifrate.

- **Generalizarea către alte primitive criptografice:** Adaptarea mecanismului de *caching* WKRP și a algoritmilor hibridi (SoL combinat cu Ensemble BKS) pentru analiza altor cifruri din clasa *lightweight* (cum ar fi Simon [1], ChaCha20 [2] sau Simeck [6]), pentru a valida dacă vulnerabilitățile descoperite sunt specifice arhitecturii ARX sau au o aplicabilitate generală.

Bibliografie

- [1] Ray Beaulieu, Douglas Shors, Jason Smith, Stefan Treatman-Clark, Bryan Weeks, and Louis Wingers. The simon and speck families of lightweight block ciphers. Technical report, Cryptology ePrint Archive, Report 2013/404, 2013.
- [2] Daniel J Bernstein. Chacha, a variant of salsa20. In *Workshop Record of SASC 2008: The State of the Art of Stream Ciphers*, volume 8, pages 1–6, 2008.
- [3] Thomas Eisenbarth, Sandeep Kumar, Christof Paar, Axel Poschmann, and Leif Uhsadel. A survey of lightweight-cryptography implementations. *IEEE Design & Test of Computers*, 24(6):522–533, 2007.
- [4] Aron Gohr. Improving attacks on round-reduced speck32/64 using deep learning. Cryptology ePrint Archive, Paper 2019/037, 2019.
- [5] Kerry McKay, Lawrence Bassham, Meltem Sönmez Turan, and Nicky Mo-
uha. Report on lightweight cryptography. Technical Report NISTIR 8114, National Institute of Standards and Technology, 2016.
- [6] Gangqiang Yang, Bo Zhu, Valentin Suder, Mark D Aagaard, and Guang Gong. The simeck family of lightweight block ciphers. In *Cryptographic Hardware and Embedded Systems – CHES 2015*, pages 307–329. Springer, 2015.
- [7] Weixi Zheng, Liu Zhang, and Zilong Wang. On the explanation and enhancement of neural-inspired differential cryptanalysis.